

```

In [87]: import math
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random
import re
from scipy.spatial.distance import cdist
import time

diversification_t_limit = 15.0
div_num_top_routes = 5
simulated_annealing_initial_temp = 45000.0
simulated_annealing_cooling_rate = 0.999

intensification_t_limit = 15.0
three_opt_max_checks = 10000

def gen_matrix(n, seed):
    """
    Bentley, J. L., & McGeoch, L. A. (1991). Engineering a TSP benchmark. In Pr
    """
    random.seed(seed)
    coords = np.random.rand(n, 2)
    dist = np.sqrt(((coords[:, None, :] - coords[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(dist, np.inf)
    return dist

def vrp_df_to_cost_matrix(customers_df):
    """
    Toth, P., & Vigo, D. (2014). Vehicle Routing: Problems, Methods, and Applic
    Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array pr
    Laporte, G. (1992). The traveling salesman problem: Overview of exact and a
    """
    coords = customers_df[["x", "y"]].to_numpy()
    dist = np.sqrt(((coords[:, None, :] - coords[None, :, :]) ** 2).sum(axis=-1))
    np.fill_diagonal(dist, np.inf) # no self-loop
    return dist

def parse_vrp_file(vrp_file):
    """
    Reinelt, G. (1991). TSPLIB-A traveling salesman problem library. ORSA Journ
    https://github.com/pdrozdowski/TSPLib.Net/tree/master retrieved 2025-11-04
    Toth, P., & Vigo, D. (2014). Vehicle Routing: Problems, Methods, and Applic
    """
    data = {"customers": [], "capacity": None}

    with open(vrp_file, "r") as f:
        section = None
        for line in f:
            line = line.strip()

            if "CAPACITY" in line:
                data["capacity"] = int(re.findall(r"\d+", line)[0])

            if line.startswith("NODE_COORD_SECTION"):
                section = "coords"
            elif line.startswith("DEMAND_SECTION"):

```

```

        section = "demand"
    elif line.startswith("DEPOT_SECTION"):
        section = "depot"

    elif section == "coords" and line and not line.startswith("DEMAND_S
        parts = line.split()
        if len(parts) == 3:
            node, x, y = map(float, parts)
            data["customers"].append({"id": int(node), "x": x, "y": y})

    elif section == "demand" and line:
        parts = line.split()
        if len(parts) == 2:
            node, demand = map(int, parts)
            for c in data["customers"]:
                if c["id"] == node:
                    c["demand"] = demand

    return pd.DataFrame(data["customers"], data["capacity"])

def parse_vrp_sol_file(sol_file):
    """
    Toth, P., & Vigo, D. (2014). Vehicle Routing: Problems, Methods, and Applic
    Helsgaun, K. (2000). An effective implementation of the Lin-Kernighan trave
    """
    routes = []
    total_cost = None
    with open(sol_file, "r") as f:
        for line in f:
            if "Route" in line:
                route = list(map(int, re.findall(r"\d+", line)))
                routes.append(route[1:])
            elif "Cost" in line or "Distance" in line:
                total_cost = float(re.findall(r"[\d.]+", line)[0])
    return routes, total_cost

def geo_distance(lat1, lon1, lat2, lon2):
    """
    Helsgaun, K. (2000). An effective implementation of the Lin-Kernighan trave
    """
    def tsplib_to_radians(value):
        # TSPLIB GEO format: DDD.MM (degrees.minutes)
        deg = int(value)
        minutes = value - deg
        return math.radians(deg + (minutes * 100.0 / 60.0))

    lat1 = tsplib_to_radians(lat1)
    lon1 = tsplib_to_radians(lon1)
    lat2 = tsplib_to_radians(lat2)
    lon2 = tsplib_to_radians(lon2)

    R = 6378.388 # tsplib Earth radius
    q1 = math.cos(lon1 - lon2)
    q2 = math.cos(lat1 - lat2)
    q3 = math.cos(lat1 + lat2)

    acos_arg = 0.5 * ((1 + q1) * q2 - (1 - q1) * q3)
    acos_arg = max(min(acos_arg, 1.0), -1.0)

    return int(R * math.acos(acos_arg) + 1)

```

```

def parse_tsp_file(filepath):
    """
    Reinelt, G. (1991). TSPLIB-A traveling salesman problem library. ORSA Journ
    https://github.com/pdrozdowski/TSPLib.Net/tree/master retrieved 2025-11-04

    Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientif
    https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance
    """
    coords = []
    edge_type = None
    with open(filepath, "r") as f:
        for line in f:
            line = line.strip()
            if line.startswith("EDGE_WEIGHT_TYPE"):
                edge_type = line.split(":")[1].strip()
            if line.startswith("NODE_COORD_SECTION"):
                break
        for line in f:
            line = line.strip()
            if line == "EOF":
                break
            parts = line.split()
            if len(parts) >= 3:
                idx, x, y = parts[:3]
                coords.append([int(idx), float(x), float(y)])
    df = pd.DataFrame(coords, columns=["id", "x", "y"]).sort_values("id")
    xy = df[["x", "y"]].values
    if edge_type == "EUC_2D":
        dist = cdist(xy, xy, metric="euclidean")
    elif edge_type == "GEO":
        raise NotImplementedError("GEO distance not implemented in this example")
    np.fill_diagonal(dist, np.inf)
    return dist, df

def parse_tsp_tour_file(tour_file, dist):
    """
    Reinelt, G. (1991). TSPLIB-A traveling salesman problem library. ORSA Journ
    https://github.com/pdrozdowski/TSPLib.Net/tree/master retrieved 2025-11-04

    Applegate, D., Bixby, R., Chvátal, V., & Cook, W. (2006). The Traveling Sal
    Helsgaun, K. (2000). An effective implementation of the Lin-Kernighan trave
    """
    tour = []
    dim = len(dist)
    in_tour_section = False
    with open(tour_file, "r", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if line.startswith("TOUR_SECTION"):
                in_tour_section = True
                continue
            if not in_tour_section:
                continue
            if line.startswith("-1") or line.startswith("EOF"):
                break
            for token in re.findall(r"-?\d+", line):
                node = int(token)

```

```

        if node == -1:
            break
        if not (1 <= node <= dim):
            continue
        tour.append(node - 1)
    print("len:", len(tour))
    if not tour:
        print(f"    no TOUR_SECTION nodes parsed in {tour_file}")
    return tour, calc_cost(tour, dist)

def calc_cost(tour, dist):
    """
    Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Trav
    """
    return sum(dist[tour[i], tour[(i+1) % len(tour)]] for i in range(len(tour)))

def simulated_annealing(dist, initial_tour, t_end):
    """
    Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by sim
    Aarts, E., & Korst, J. (1988). Simulated Annealing and Boltzmann Machines:
    Croes, G. A. (1958). A method for solving traveling-salesman problems. Oper
    Beale, R., & Demir, N. (2020). Practical implementation of simulated anneal
    """
    current = list(range(len(dist))) if initial_tour is None else initial_tour.
    best = current.copy()
    best_cost = calc_cost(best, dist)
    current_cost = best_cost
    temp = simulated_annealing_initial_temp

    improvements = 0
    while time.time() < t_end and temp > 1e-6:
        i, j = sorted(random.sample(range(len(current)), 2))
        new_tour = current[:i+1] + current[i+1:j+1][::-1] + current[j+1:]
        new_cost = calc_cost(new_tour, dist)
        delta = new_cost - current_cost

        if delta < 0 or random.random() < np.exp(-delta / temp):
            current = new_tour
            current_cost = new_cost
            if new_cost < best_cost:
                improvements += 1
                best = new_tour.copy()
                best_cost = new_cost

        temp *= simulated_annealing_cooling_rate

    return best, best_cost, improvements

def three_opt(dist, initial_tour, t_end, max_checks):
    """
    Lin, S. (1965). Computer solutions of the traveling salesman problem. Bell
    Reinelt, G. (1994). The traveling salesman: Computational solutions and ben
    Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Trav
    """

    def three_opt_candidate(tour, i, j, k, case):

```

```

A = tour[:i+1]
B = tour[i+1:j+1]
C = tour[j+1:k+1]
D = tour[k+1:]

if case == 0:
    return A + B[::-1] + C + D
if case == 1:
    return A + C + B + D
if case == 2:
    return A + C[::-1] + B[::-1] + D
if case == 3:
    return A + C[::-1] + B + D
if case == 4:
    return A + B[::-1] + C + D
if case == 5:
    return A + C + B[::-1] + D
return A + C + B + D

n = len(dist)
tour = list(range(n)) if initial_tour is None else initial_tour.copy()
best_cost = calc_cost(tour, dist)

improvements = 0
while time.time() < t_end:
    improved = False
    checks = 0
    while checks < max_checks and time.time() < t_end:
        checks += 1
        if n < 4:
            break # too small for 3-opt

        i = random.randrange(0, n - 2)

        j_start = i + 2
        j_end = n - 1
        if j_start >= j_end:
            continue
        j = random.randrange(j_start, j_end)

        k_start = j + 2
        k_end = n + (i > 0)
        if k_start >= k_end:
            continue
        k = random.randrange(k_start, k_end)

        k = k % n # normalize

        for case in range(7):
            candidate = three_opt_candidate(tour, i, j, k, case)
            cand_cost = calc_cost(candidate, dist)
            if cand_cost < best_cost - 1e-12:
                tour = candidate
                best_cost = cand_cost
                improved = True
                improvements += 1
                break
        if improved:
            break
    if not improved:
        break

```

```

    return tour, best_cost, improvements

def solve(dist):
    print("  CesiCDP:")
    start_time = time.time()
    t_end_div = start_time + diversification_t_limit
    t_end_int = t_end_div + intensification_t_limit
    best_routes = [(None, np.inf)] * 5

    def random_starts(n):
        rand_route = list(range(n))
        random.shuffle(rand_route)
        return rand_route

    print("    1/2 Diversification phase SA... ", end="")
    div_improvements = 0
    while time.time() < t_end_div:
        path, cost, impv = simulated_annealing(dist, random_starts(len(dist)),
        div_improvements += impv
        if cost >= max(best_routes, key=lambda x: x[1])[1]:
            continue # no improvement
        if any(np.array_equal(path, p) for p, _ in best_routes):
            continue # same route
        best_routes.append((path.copy(), cost))
        best_routes = sorted(best_routes, key=lambda x: x[1])[:div_num_top_routes]
    print(f"{div_improvements} best routes reduced to:")
    for idx, (_, i_cost) in enumerate(best_routes):
        if i_cost != np.inf:
            print(f"    Top {idx+1}: {i_cost}")

    best_path, best_cost = best_routes[0]
    int_improvements = 0
    print("    2/2 Intensification phase 3-OPT... ", end="")
    for (i_path, _) in best_routes:
        path, cost, impv_3opt = three_opt(dist, i_path, t_end_int, three_opt_max)
        int_improvements += impv_3opt
        if cost < best_cost:
            best_path = path
            best_cost = cost
    print(f"with {int_improvements} improvements.")

    return best_path, best_cost, time.time() - start_time

def cesi_summarize(elapsed, benchmark, cost):
    # cycle = path + [path[0]]
    print(f"    Time elapsed:      {elapsed:.3f} seconds")
    print(f"    Solution cost:      {cost}")
    print(f"    Solution score:      {(benchmark / cost * 100):.2f}%\n")

def vrp_summarize(vrp_file, sol_file):
    customers_df, capacity = parse_vrp_file(vrp_file)
    routes, total_cost = parse_vrp_sol_file(sol_file)

    print("  VRPLib:")
    print(f"    Number of customers: {len(customers_df)}")
    print(f"    Vehicle capacity:    {capacity}")
    print(f"    Total demand:        {customers_df['demand'].sum()}")
    print(f"    Number of routes (veh): {len(routes)}")
    print(f"    Total solution cost:  {total_cost}")
    return total_cost

```

```

def plot_tsp_route(coords_df, path, title="TSP Route"):
    """
    Reinelt, Gerhard. TSPLIB: A Traveling Salesman Problem Library. ORSA Journal
    to be refined for clarify...
    """
    xy = coords_df[["x", "y"]].to_numpy()

    # close the route by returning to the start
    tour = path + [path[0]]

    plt.figure(figsize=(10, 7))
    plt.plot(xy[tour, 0], xy[tour, 1], linewidth=0.8)
    plt.scatter(xy[:, 0], xy[:, 1], s=5)
    plt.title(title)
    plt.tight_layout()
    plt.show()

print("VRP B-n31-k5:")
customers_df_B, _ = parse_vrp_file("B-n31-k5.vrp")
dist_B = vrp_df_to_cost_matrix(customers_df_B)
bench_B = vrp_summarize("B-n31-k5.vrp", "B-n31-k5.sol")
path_B, cost_B, elapsed_B = solve(dist_B)
cesi_summarize(elapsed_B, bench_B, cost_B)

print("VRP A-n32-k5:")
customers_df_A, _ = parse_vrp_file("A-n32-k5.vrp")
dist_A = vrp_df_to_cost_matrix(customers_df_A)
bench_A = vrp_summarize("A-n32-k5.vrp", "A-n32-k5.sol")
path_A, cost_A, elapsed_A = solve(dist_A)
cesi_summarize(elapsed_A, bench_A, cost_A)

print("VRP M-n101-k10:")
customers_df_M, _ = parse_vrp_file("M-n101-k10.vrp")
dist_M = vrp_df_to_cost_matrix(customers_df_M)
bench_M = vrp_summarize("M-n101-k10.vrp", "M-n101-k10.sol")
path_M, cost_M, elapsed_M = solve(dist_M)
cesi_summarize(elapsed_M, bench_M, cost_M)

"""
Reinelt, G. (1991). TSPLIB-A traveling salesman problem library. ORSA Journal o
Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Travelin
"""
tsp_known_optimums = {
    "berlin52": 7542,
    "ch130": 6110,
    "rat575": 6773,
    "pr1002": 259045,
    "pr2392": 378032,
    "usa13509": 19982859,
}

print("TSP berlin52:")
(dist_52, _), bench_52 = parse_tsp_file("berlin52.tsp"), tsp_known_optimums['be
print(" TSplib:           known optimum:", bench_52)
path_52, cost_52, elapsed_52 = solve(dist_52)
cesi_summarize(elapsed_52, bench_52, cost_52)

```

```

print("TSP ch130:")
(dist_130, _), bench_130 = parse_tsp_file("ch130.tsp"), tsp_known_optimums['ch1
print("  TSPLib:          known optimum:", bench_130)
path_130, cost_130, elapsed_130 = solve(dist_130)
cesi_summarize(elapsed_130, bench_130, cost_130)

print("TSP rat575:")
(dist_575, _), bench_575 = parse_tsp_file("rat575.tsp"), tsp_known_optimums['ra
print("  TSPLib:          known optimum:", bench_575)
path_575, cost_575, elapsed_575 = solve(dist_575)
cesi_summarize(elapsed_575, bench_575, cost_575)

print("TSP pr1002:")
(dist_1002, _), bench_1002 = parse_tsp_file("pr1002.tsp"), tsp_known_optimums['
print("  TSPLib:          known optimum:", bench_1002)
path_1002, cost_1002, elapsed_1002 = solve(dist_1002)
cesi_summarize(elapsed_52, bench_1002, cost_1002)

print("TSP pr2392:")
(dist_2392, coords_2392), bench_2392 = parse_tsp_file("pr2392.tsp"), tsp_known_
print("  TSPLib:          known optimum:", bench_2392)
path_2392, cost_2392, elapsed_2392 = solve(dist_2392)
cesi_summarize(elapsed_2392, bench_2392, cost_2392)
plot_tsp_route(coords_2392, path_2392, title="TSP pr2392 – Best Route Found")

print("TSP usa13509:")
(dist_2392, _), bench_2392 = parse_tsp_file("usa13509.tsp"), tsp_known_optimums
print("  TSPLib:          known optimum:", bench_2392)
path_2392, cost_2392, elapsed_2392 = solve(dist_2392)
cesi_summarize(elapsed_2392, bench_2392, cost_2392)

```

VRP B-n31-k5:

VRPLib:

Number of customers: 31
Vehicle capacity: 100
Total demand: 412
Number of routes (veh): 5
Total solution cost: 672.0

CesiCDP:

1/2 Diversification phase SA... 11922 best routes reduced to:

Top 1: 287.31613407724217
Top 2: 287.31613407724217
Top 3: 287.31613407724217
Top 4: 287.31613407724217
Top 5: 287.31613407724217

2/2 Intensification phase 3-OPT... with 0 improvements.

Time elapsed: 16.557 seconds
Solution cost: 287.31613407724217
Solution score: 233.89%

VRP A-n32-k5:

VRPLib:

Number of customers: 32
Vehicle capacity: 100
Total demand: 410
Number of routes (veh): 5
Total solution cost: 784.0

CesiCDP:

1/2 Diversification phase SA... 10068 best routes reduced to:

Top 1: 467.1054999049506
Top 2: 467.10549990495065
Top 3: 467.10549990495065
Top 4: 467.10549990495065
Top 5: 467.10549990495065

2/2 Intensification phase 3-OPT... with 0 improvements.

Time elapsed: 16.569 seconds
Solution cost: 467.1054999049506
Solution score: 167.84%

VRP M-n101-k10:

VRPLib:

Number of customers: 101
Vehicle capacity: 200
Total demand: 1810
Number of routes (veh): 10
Total solution cost: 820.0

CesiCDP:

1/2 Diversification phase SA... 18410 best routes reduced to:

Top 1: 524.2972038250375
Top 2: 532.7237214000102
Top 3: 533.9833824145259
Top 4: 534.7821722739827
Top 5: 535.8271595181325

2/2 Intensification phase 3-OPT... with 43 improvements.

Time elapsed: 22.507 seconds
Solution cost: 508.80160694461824
Solution score: 161.16%

TSP berlin52:

TSPLib: known optimum: 7542

CesiCDP:

1/2 Diversification phase SA... 13943 best routes reduced to:

Top 1: 7735.798385217678
Top 2: 7777.332311275912
Top 3: 7780.413236734768
Top 4: 7800.275952255715
Top 5: 7810.6540897625055
2/2 Intensification phase 3-OPT... with 3 improvements.
Time elapsed: 17.511 seconds
Solution cost: 7735.798385217678
Solution score: 97.49%

TSP ch130:

TSPLib: known optimum: 6110

CesiCDP:

1/2 Diversification phase SA... 17686 best routes reduced to:

Top 1: 7081.456547270797
Top 2: 7094.075531938375
Top 3: 7103.198982239593
Top 4: 7114.344033149121
Top 5: 7116.55669270178

2/2 Intensification phase 3-OPT... with 126 improvements.

Time elapsed: 30.000 seconds
Solution cost: 6429.659946235566
Solution score: 95.03%

TSP rat575:

TSPLib: known optimum: 6773

CesiCDP:

1/2 Diversification phase SA... 13287 best routes reduced to:

Top 1: 22069.105684955553
Top 2: 22452.202623737972
Top 3: 22637.495026344008
Top 4: 23005.596981124174
Top 5: 23099.86667589298

2/2 Intensification phase 3-OPT... with 601 improvements.

Time elapsed: 30.000 seconds
Solution cost: 12819.486659858345
Solution score: 52.83%

TSP pr1002:

TSPLib: known optimum: 259045

CesiCDP:

1/2 Diversification phase SA... 11691 best routes reduced to:

Top 1: 1524829.0120613503
Top 2: 1578464.4394538626
Top 3: 1590846.2980073052
Top 4: 1593896.704995275
Top 5: 1611122.4503948365

2/2 Intensification phase 3-OPT... with 792 improvements.

Time elapsed: 17.511 seconds
Solution cost: 980256.2142416055
Solution score: 26.43%

TSP pr2392:

TSPLib: known optimum: 378032

CesiCDP:

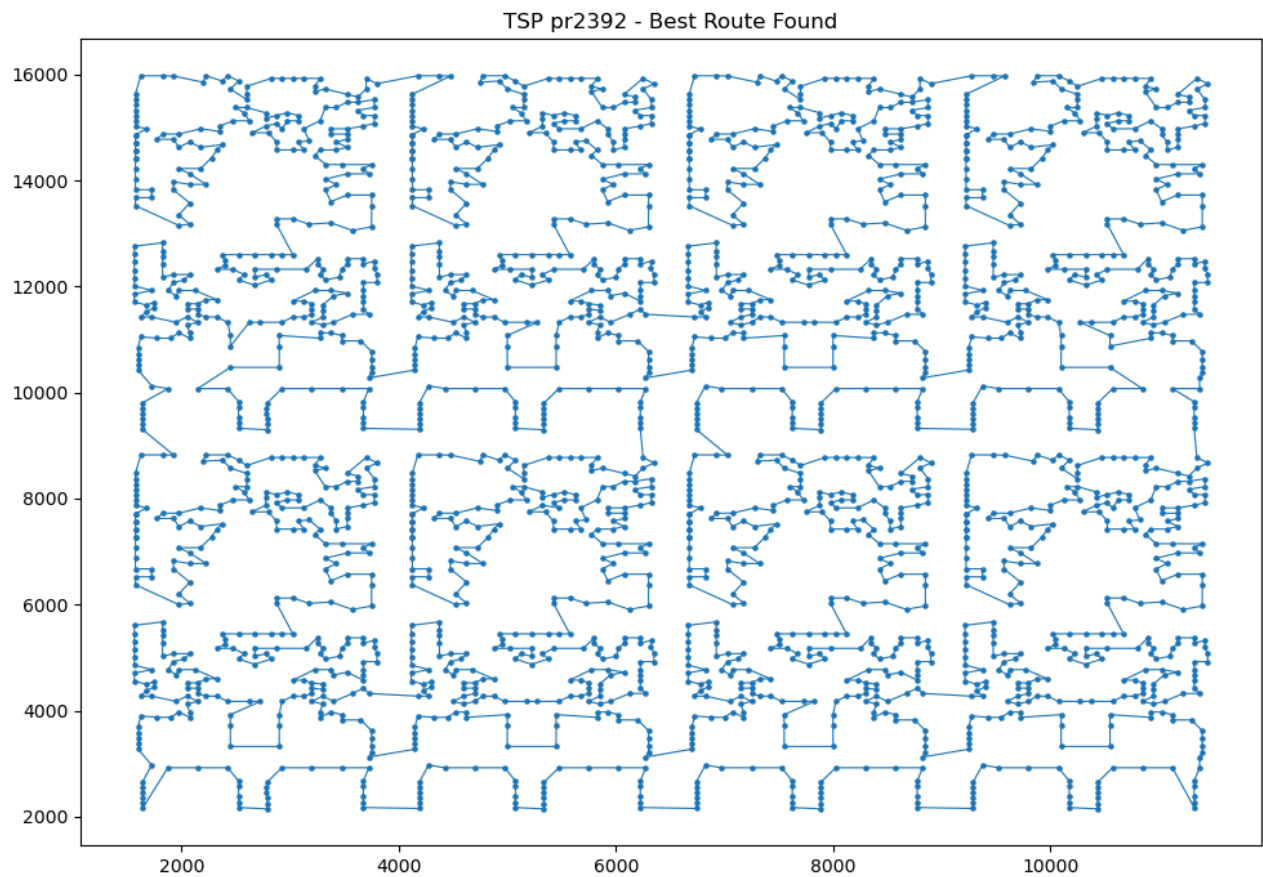
1/2 Diversification phase SA... 8331 best routes reduced to:

Top 1: 5523413.612080276
Top 2: 5538002.471602721
Top 3: 13008266.537825802

2/2 Intensification phase 3-OPT... with 1013 improvements.

Time elapsed: 30.002 seconds

Solution cost: 378062.82619143813
Solution score: 99.99%



TSP usa13509:

TSPLib: known optimum: 19982859

CesiCDP:

1/2 Diversification phase SA... 3655 best routes reduced to:

Top 1: 1819250840.7220027

2/2 Intensification phase 3-OPT... with 1579 improvements.

Time elapsed: 30.029 seconds

Solution cost: 1590833038.0920846

Solution score: 1.26%

In []: